Roma Tre

iNGEGNERIA
CIVILE, INFORMATICA E DELLE TECNOLOGIE AERONAUTICHE

Sequential Testing Problem

Decision Support Systems and Analytics

Riccardo Moccia

Overview

- Descrizione Problema
- Complessità(perché servono euristiche)
- Metodo esatto
- Simulated annealing
- Euristiche greedy
- Implementazione
- Test
- Risultati
- Conclusioni e idee per sviluppi futuri

Descrizione problema

- Input
  - Un DAG $G=(V,E)$
  - Per ogni nodo/test $i \in V$:
    - costo $c(i)>0$
    - probabilità di successo $p(i) \in (0,1)$
- Vincoli
  - Se c'è un arco $u \rightarrow v$, allora u deve comparire prima di v (ordine topologico)
- Output
  - Un ordine topologico π che minimizza il costo atteso

Applicazioni sequential testing

- Assunzione operativa
  - I test vengono eseguiti in sequenza
  - Ci si ferma al primo fallimento (se un test fallisce, non ha senso proseguire)
- Alcuni ambiti applicativi:
  - Diagnosi: appena trovi un problema puoi interrompere
  - Controllo qualità: stop alla prima non conformità
  - Pipeline/validazioni: fallimento blocca step successivi

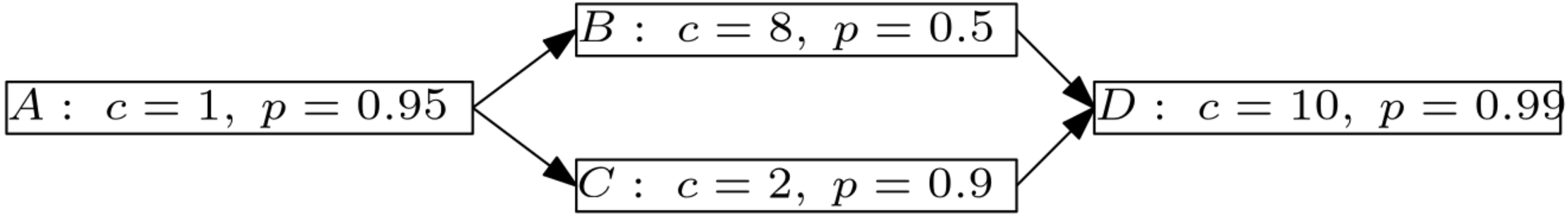
Funzione obiettivo: costo atteso

- L'obiettivo è quello di minimizzare il costo atteso
- Costo atteso dell'ordine $\pi = (v_1, \dots, v_n)$

$$E[C(\pi)] = \sum_{k=1}^{n} C(v_k) \prod_{J<k} p(v_J)$$

- Per ogni nodo/test $i \in V$:
  - $\prod_{J<k} p(v_J)$ = probabilità di arrivare al test k senza essersi fermati prima

Esempio



- Ordine A->B->C->D
  - $E[C(A,B,C,D)] = \ c_a + p_a c_b + (p_a p_b) c_c + (p_a p_b p_c) c_d = 13.825$
- Ordine A->C->B->D
  - $E[C(A,C,B,D)] = \ c_a + p_a c_c + (p_a p_c) c_b + (p_a p_c p_b) c_d = 14.015$

Complessità(perché servono euristiche)

- Se non ci sono vincoli (test indipendenti e nessun DAG), l'ordine ottimo è "semplice"
- Introducendo vincoli di precedenza generali (ordine parziale/DAG), trovare l'ordine che minimizza il costo atteso diventa NP-hard [Berend et al. 14]
  - Proof by reduction from the optimal linear arrangement

Metodo esatto

- Idea
  - Genero tutti gli ordinamenti topologici del DAG
  - Calcolo $E[C(\pi)]$ per ciascuno
  - Tengo il migliore
- Implementazione tramite recursive backtracking
  - Insieme di nodi candidati = quelli con indegree 0 tra i rimanenti
  - Ricorsione su ogni possibile scelta
  - Attenzione: esplode rapidamente → usato solo per pochi nodi

Simulated annealing

- Parto da un ordinamento topologica casuale
- Mossa utilizzata
  - swap di due posizioni, accettato solo se l'ordine resta topologico
- Regola di accettazione
  - Se migliora: accetto
  - Se peggiora: accetto con probabilità

$$\Pr(\text{accept}) = \mathrm{e}^{\left(-\frac{\Delta}{T}\right)}$$

Parametri simulated annealing

- Metodo pratico per settare T_start nel SA [Ben-Ameur 04]
  1. Prendi un ordine topologico casuale
  2. Prova tante mosse try_swap a caso
  3. Per le mosse valide, calcola delta = neighbor_cost - current_cost
  4. Raccogli solo i delta > 0
  5. Prendi la mediana d
  6. Imposta T_start = - d / ln(p0) con p0 tra 0.7 e 0.9 (0.8)

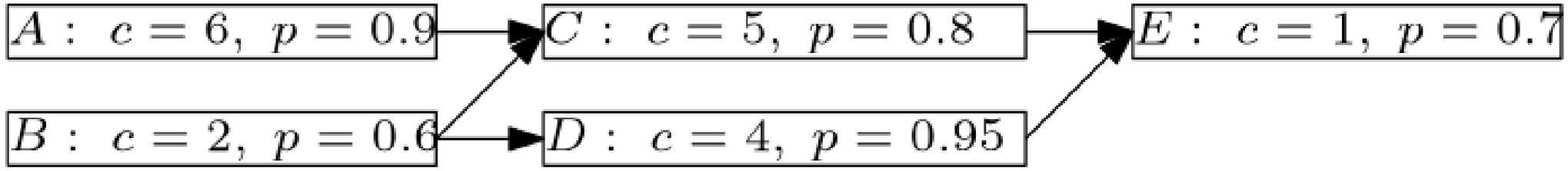
Parametri simulated annealing

- Utilizzando ragionamenti analoghi, i parametri sono stati impostati nel seguente modo:
  - T_start=50.0
  - T_end=1.0
  - alpha=0.99
  - iters_per_T=200

Euristiche greedy(costruzione incrementale)

- Schema
  - Mantengo i nodi “disponibili” (indegree 0)
  - A ogni passo scelgo il nodo con score minimo: score1(i)=$c(i)/p(i)$ score2(i)=$c(i)/(1-p(i))$

Esempio euristica greedy1



- Esecuzione Greedy
  - Calcolo score1: score(A)=6.67 score(B)=3.33 score(C)=6.25 score(D)=4.21 score(E)=1.43
  - Step 0 disponibili: {A, B} → min score = B
  - Step 1 disponibili: {A, D} (D sblocca perché dipende solo da B) → min = D
  - Step 2 disponibili: {A} → A
  - Step 3 disponibili: {C} (C sblocca perché A e B sono fatti) → C
  - Step 4 disponibili: {E} (E sblocca perché C e D sono fatti) → E

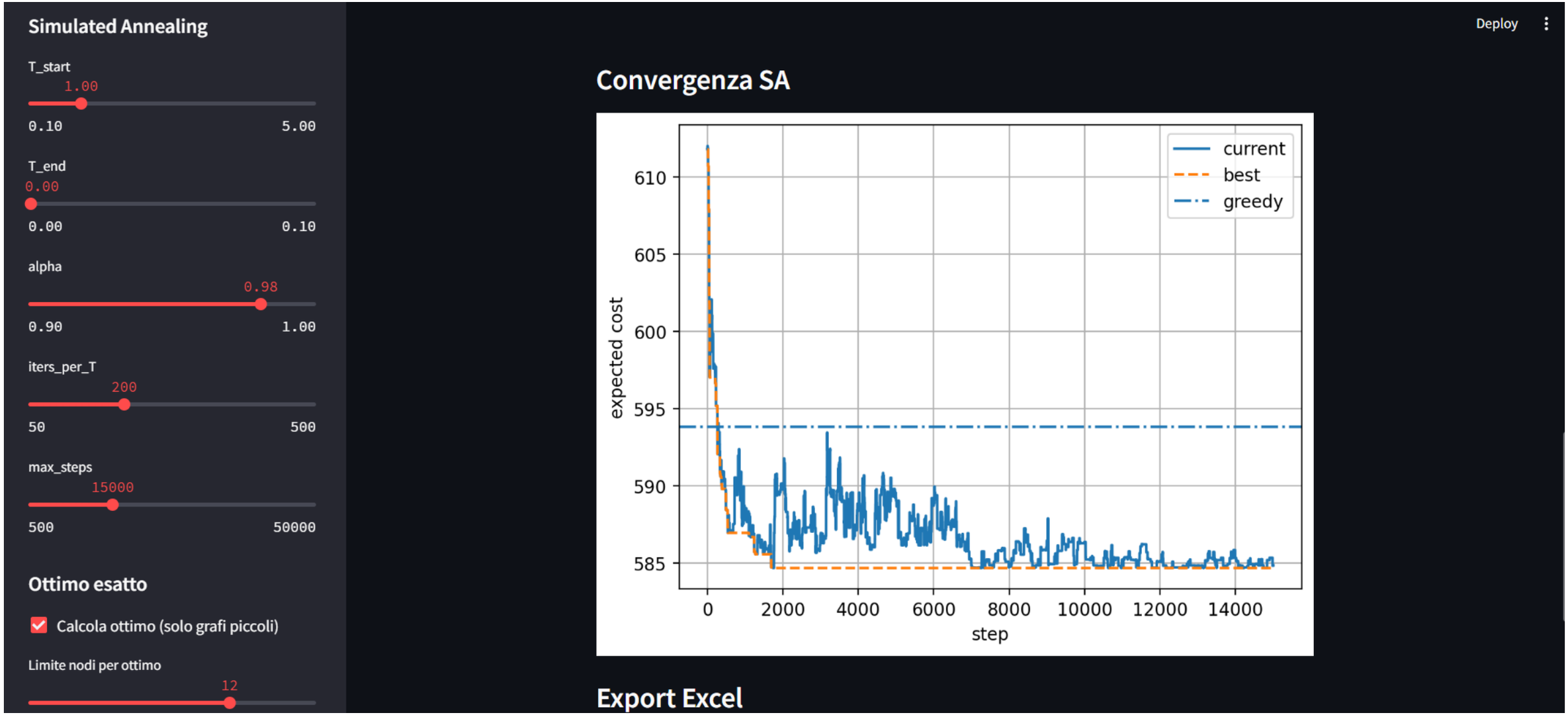
Implementazione

- Librerie
  - *Networkx* per DAG e topological sort
  - *Streamlit* per demo interattiva
  - *Pandas* + export Excel per risultati batch
  - *Matplotlib* per plot grafico

Implementazione

- Struttura file
  - *problem.py*: modello/ expected_cost()/ mosse valide (try_swap)
  - *exact.py*: enumerazione di tutti i topological sort(backtracking)
  - *heuristics.py*: greedy + simulated annealing
  - *app.py*: UI Streamlit + grafici + export
  - *batch_run.py*: esecuzione su cartella /test e salvataggio batch_results.xlsx

Interfaccia applicazione



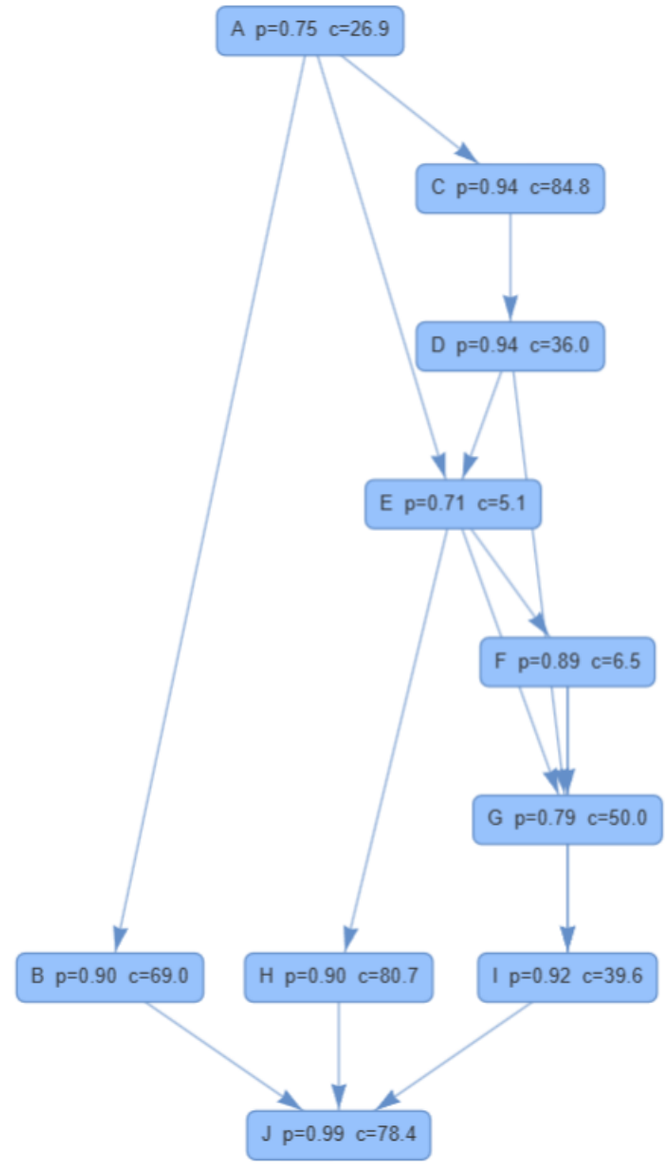
Dataset di test(120 istanze)

- Struttura complessiva
  - Totale: 120 istanze di DAG
  - 3 gruppi per dimensione
    - $N = 10, |E| = 15 \rightarrow 40 \text{ istanze}$
    - $N = 15, |E| = 23 \rightarrow 40 \text{ istanze}$
    - $N = 20, |E| = 30 \rightarrow 40 \text{ istanze}$
- In ciascun gruppo
  - 10 topologie diverse(stessa struttura archi, pesi diversi)
  - Per ogni topologia: 4 assegnazioni di (p,c) con intervalli diversi
  - Totale per gruppo: 10x4= 40 istanze

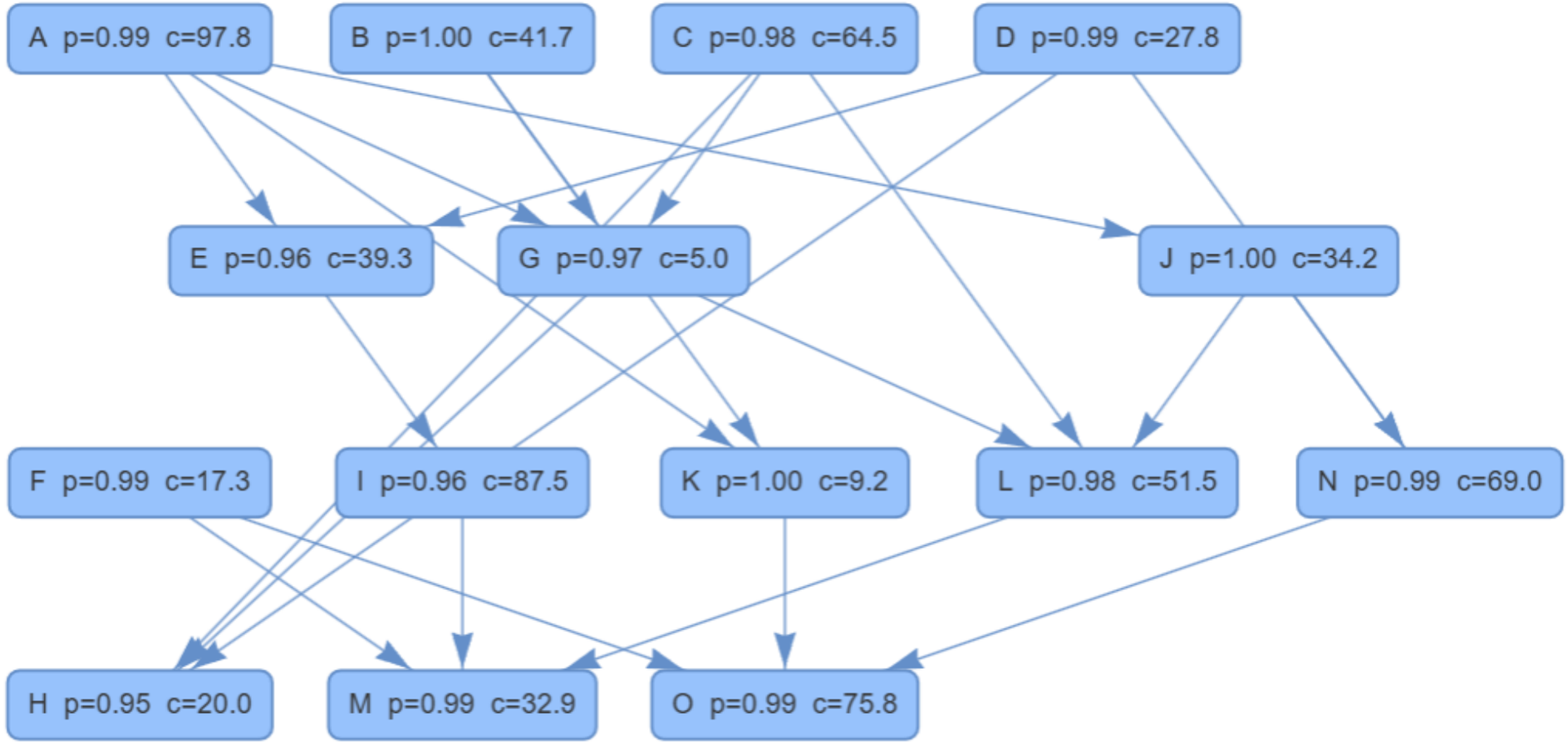
Dataset di test(120 istanze)

- Per ogni topologia, si generano 4 varianti assegnando a ogni nodo:
  - I1: $p \sim U[0.95,1]$ $c \sim U[1,100]$
  - I2: $p \sim U[0.70,1]$ $c \sim U[10,1000]$
  - I3: $p \sim U[0.95,1]$ $c \sim U[10,1000]$
  - I4: $p \sim U[0.70,1]$ $c \sim U[1,100]$
    - $40 = 10\ topologie \times 4\ intervalli"\ ripetuto\ per\ N = 10/15/20$

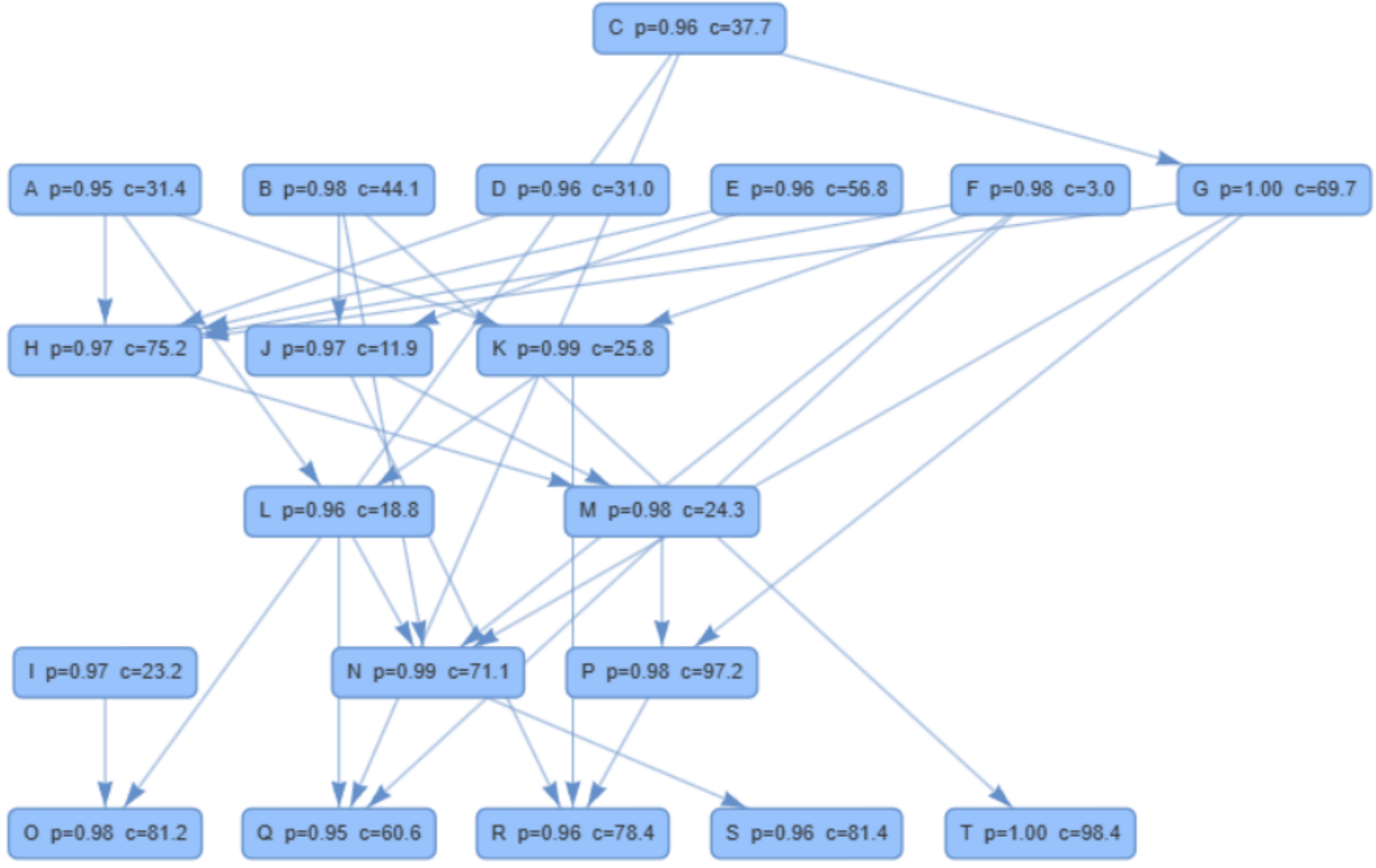
Esempio grafo primo gruppo(n=10)



Esempio grafo secondo gruppo(n=15)



Esempio grafo terzo gruppo(n=20)



Risultati(1)

- Greedy2 domina Greedy1
  - Miglioramento di ~3.6% in media
  - L'effetto cresce con N (circa -1.7% a N=10 → -4.9% a N=20)

Risultati(2)

- SA è quasi sempre la soluzione migliore, soprattutto nell'intervallo

Intervallo	SA vs Greedy2 (media)	SA vs Greedy1 (media)
I1 (p alte, costi bassi)	~0%	-0.6%
I2 (p 0.7–1, costi 10–1000)	**-4.16%**	**-9.68%**
I3 (p alte, costi alti)	-0.48%	-1.0%
I4 (p 0.7–1, costi 1–100)	-1.02%	-8.69%

  - Possibile interpretazione: quando molte probabilità non sono vicinissime a 1 (I2/I4) e/o i costi sono grandi e variabili (I2), la scelta dell'ordine impatta molto il costo atteso → il simulated annealing si rivela molto efficace

Risultati(3)

- Su N=10: SA è praticamente ottimo

Algoritmo	Gap medio vs OPT	% volte esattamente OPT
Greedy1	+2.56%	15%
Greedy2	+0.69%	65%
SA	**+0.0067%**	**92.5%**

Risultati(4)

- SA è molto più lento dei greedy
  - Tempi medi Greedy: ~0.05–0.08 ms
  - Tempi medi SA: ~83 ms (N10)| 99 ms (N15) 119 ms (N20)

Risultati(5)

- SA spesso trova il valore che restituirà come valore finale abbastanza presto
  - Dalla colonna *sa_time_to_final_best_frac*:
  - mediana ≈ 0.38 → in metà casi il best finale arriva entro il 38% del tempo
  - primo quartile ≈ 0.07 → in un 25% dei casi arriva entro il 7%
  - Questo è utile per motivare early stopping o budget time-based.

Idee per sviluppi futuri

- Migliorare la ricerca
  - Mosse più ricche dello swap: aggiungere la mossa di insert
  - Multi-start SA / restart
  - Tabu Search(+ altre infinite metaeuristiche…)
- Modello più realistico
  - Probabilità dipendenti/condizionate
  - Stop non al primo fallimento (k-out-of-n, soglie, ecc.)
  - Vincoli aggiuntivi (es. finestre temporali)

Bibliography and further readings

- [Ben-Ameur 04] W. Ben-Ameur “Computing the Initial Temperature of Simulated Annealing”. Computational Optimization and Applications, 29, 369–385, 2004
- [Berend et al. 14] Optimal ordering of independent tests with precedence constraints, Discrete Applied Mathematics, 2014